# Exploring fine tuning for semantic correspondence on transformer based models

Mohammadreza Babaei
Politecnico di Torino (PoliTo)
https://github.com/mohammadreza-babaei/semantic-correspondence

## Abstract

*This work benchmarks DINOv2, DINOv3, and SAM for dense semantic correspondence on the SPair-71k dataset. We introduce a coarse-to-fine pipeline utilizing a Window Soft-Argmax strategy to mitigate patch-based quantization errors. By partially fine-tuning deep transformer blocks, we successfully specialize generic representations into geometry-aware descriptors capable of distinguishing symmetric parts. Our experiments show that DINOv2 combined with photometric augmentation achieves the best performance, reaching a PCK@0.10 of 80.35%. We further explore efficiency through lightweight architectures (TinyViT vs. TinySAM) and Parameter-Efficient Fine-Tuning (PEFT). While LoRA reduces storage requirements by over 97% (0.57MB), we find it struggles to capture the high-rank transformations required for this task compared to standard fine-tuning.*

## 1. Introduction

Establishing dense correspondences between images depicting different instances of the same object category, known as semantic correspondence, remains a pivotal challenge in computer vision [6, 9]. Semantic correspondence is about finding matching points or pixels between two images that show different objects from the same category (e.g example, matching the leg of one horse to the leg of another animal). This is more difficult than tasks like stereo matching or optical flow, because in those kind of tasks the images usually come from the same scene. Here, the objects can look very various, they can have different poses, and the background can also be totally different.
Unlike standard stereo matching or optical flow, which deal with the same scene across viewpoints or time, semantic correspondence requires handling large intra-class variations, significant deformations, and diverse background clutter [9]. Successfully solving this task unlocks a wide range of downstream applications, including exemplar-based image editing [9], object co-segmentation, and cross-domain style transfer [6].

In this work, we compare DINOv2, DINOv3, and SAM for semantic correspondence. The key idea is to see how well these models handle semantic matching, and also whether they can keep fine geometric details correct (for example, not confusing left and right symmetric parts) [6]. We want to distinguish the differences between self-supervised models (DINO) and a segmentation-based model (SAM), and how each one affects the quality of the features for this task. In particular, the DINO family of models has set a strong baseline, showing that patch-level features from **Vision Transformers (ViT)** [1] naturally cluster by semantic parts. In this work, we investigate and evaluate the efficacy of three distinct foundation model architectures for the task of semantic correspondence:

- **DINOv2**: A robust self-supervised model that scales the DINO objective to create all-purpose visual features, known for its stability and high performance on dense prediction tasks [3].
- **DINOv3**: The latest iteration in the DINO lineage, designed to further refine feature granularity and scalability, potentially addressing limitations in previous self-supervised representations [5].
- **Segment Anything Model (SAM)**: A promptable segmentation foundation model trained on the massive SA-1B dataset. While designed for segmentation, SAM's image encoder learns to respect object boundaries and part structures, making it a compelling yet under-explored candidate for dense semantic matching [2].

We analyze these models in the context of recent findings that suggest that while foundation models excel at high-level semantics, they often struggle with fine-grained geometric awareness, such as distinguishing left vs. right symmetric parts [9]. By benchmarking DINOv2, DINOv3, and SAM, we aim to disentangle the contributions of self-supervised objective functions versus segmentation-driven supervision in learning geometrically consistent and semantically rich feature descriptors.

## 2. Related Work

### Semantic Correspondence and Discriminative Features

Modern semantic correspondence has shifted from learning specialized matching networks to a "backbone-centric" paradigm, where performance relies heavily on the quality of pre-trained feature extractors. Currently, Self-Supervised Learning (SSL) serves as the gold standard for this task. DINO and DINOv2 have demonstrated that discriminative self-supervised objectives yield patch-level features with strong semantic alignment across object instances. Building on this, DINOv3 addresses the feature degradation often observed in large-scale ViTs by introducing Gram anchoring, explicitly regularizing the latent space to preserve the fine-grained spatial granularity required for dense matching.

### Structural Awareness and the Limits of SSL

Despite their semantic robustness, standard SSL models exhibit critical limitations in dense structural understanding. Zhang et al. (2024) [9] identified that models like DINOv2 often lack fine-grained spatial distinctiveness, frequently failing to differentiate between visually similar but spatially distinct object parts due to the translation-invariant nature of their training objectives. While DINO excels at high-level semantic identity ("this is a cat"), it captures less low-level structural detail compared to other architectures [8]. They proposed that optimal dense correspondence requires fusing these abstract semantic features with descriptors that preserve local shape and structure .

### Emergent Features in Generative and Task-Specific Models

To address these structural deficits, recent research has looked toward generative foundation models. Tang et al. (2023) [6] demonstrated "emergent correspondence" in text-to-image diffusion models, showing that internal representations from the Stable Diffusion U-Net (specifically DIFT) contain dense spatial information that outperforms weakly-supervised baselines [6]. Parallel to this generative trend, we investigate task-specific discriminative models like the Segment Anything Model (SAM). Unlike DINO's self-supervision, SAM is trained with explicit mask supervision to resolve object boundaries and topology. In this work, we analyze whether the specialized encoders of SAM and the granular features of DINOv3 can bridge the structural gap identified by Zhang et al. [9], offering a more efficient alternative to heavy diffusion-based features.

## 3. Feature Extraction

To leverage the semantic priors of foundation models for correspondence, we adopt three architectures: DINOv2, DINOv3, and SAM. All models employ a Vision Transformer (ViT) backbone, but differ in their tokenization strategies, positional encodings, and spatial representations. For fair comparison, input images are resized so that their dimensions are multiples of the patch size (p=14 for DINOv2, p=16 for DINOv3 and SAM), targeting a standard resolution of approximately $512 \times 512$ pixels (518 for DINOv2 to match its pre-training crop). For DINOv2, we follow the standard ViT feature extraction pipeline by patch-embedding the image into a sequence of tokens, discarding the `[CLS]` token, and reshaping the remaining patch tokens into a spatial feature grid; most transformer blocks are frozen, with only the last $k$ blocks and the final LayerNorm fine-tuned for correspondence. DINOv3 extends this design by replacing absolute positional embeddings with Rotary Positional Embeddings (RoPE), requiring a modified forward pass, and introduces additional register tokens, which are explicitly filtered out together with the `[CLS]` token to retain only patch-level features for dense matching. In contrast, SAM preserves spatial structure throughout the backbone by maintaining channel-last feature maps $(B, H, W, C)$ and incorporates a specialized neck module following the transformer blocks; we extract features after this neck, yielding compact, spatially consistent dense representations well-suited for correspondence.

## 4. Methodology

### 4.1. Keypoint Refinement

To achieve sub-patch localization accuracy, we implement a two-stage *coarse-to-fine* prediction mechanism. Standard Vision Transformer features are inherently limited by their patch resolution (e.g., $14 \times 14$ pixels), which can be too coarse for precise semantic correspondence. We address this using a window-based refinement strategy.

**Global Coarse Prediction.** First, we compute the global similarity map $S \in \mathbb{R}^{H \times W}$ between the source keypoint feature $f_s$ and the entire grid of target features $F_t$. We apply a spatial Argmax operation to obtain the discrete coordinates of the best match $\hat{\mathbf{p}}_{global}$:

$$\hat{\mathbf{p}}_{global} = \underset{x}{\text{argmax}} \left( S(x) \right) \tag{1}$$

Note that this discrete operation is non-differentiable and is used primarily for inference to initialize the local window. During training, we replace this with a Global Soft-Argmax operation to ensure differentiability, allowing gradients to propagate through the network.

**Local Window Refinement.** Global predictions can be susceptible to multi-modal distributions (e.g., confusing left vs. right symmetric parts) or background clutter, a limitation highlighted in recent geometry-aware correspondence

research [9]. To mitigate this, we refine the prediction by isolating a local region around the coarse estimate.
We define a window $\mathcal{W}$ of size $h_w \times w_w$ centered at the integer coordinates $\hat{\mathbf{p}}_{global}$. We then compute the Soft-Argmax strictly within this local window to recover sub-pixel precision:

$$\hat{\mathbf{p}}_{refined} = \sum_{x \in \mathcal{W}} x \cdot \text{Softmax}\left(\frac{S(x)}{\tau}\right) \tag{2}$$

where $\tau$ is a temperature parameter controlling the sharpness of the distribution. This "zoom-in" step effectively filters out distant distractors while allowing gradients to flow through the local feature alignment.

### 4.2. Feature Similarity Metric

Given a set of annotated key-point pairs from source images $\mathcal{P}^s = \{p_1^s, ..., p_n^s\}$ and target images $\mathcal{P}^t = \{p_1^t, ..., p_n^t\}$ and the post-processed source $\tilde{\mathbf{F}}^s$ and $\tilde{\mathbf{F}}^t$ target features, we rely on the **cosine similarity** with $L_2$ normalization

$$\text{sim}(u, v) = \frac{u \cdot v}{||u||_2 ||v||_2} \tag{3}$$

The global similarity map for a keypoint $p_i \in \mathbb{R}^{H \times W}$ is then computed via:

$$\text{sim}(\tilde{\mathbf{F}}^s(p_i), \tilde{\mathbf{F}}^t(p_i)) \tag{4}$$

This ensures that the matching process relies purely on the angular alignment of the semantic features, making it robust to scaling variations in the feature space.

### 4.3. Loss Function

We optimize the model using a coordinate regression loss similar to the keypoint-alignment objectives used in previous correspondence works [9]. The objective is to minimize the Mean Squared Error (MSE) between the predicted keypoints and the ground-truth annotations:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} ||\hat{\mathbf{p}}_{global}^{(i)} - \mathbf{p}_{gt}^{(i)}||_2^2 \tag{5}$$

where $N$ is the number of visible key points in the image pair. This direct coordinate supervision encourages the feature encoder to learn semantic consistency that is geometrically accurate.

### 4.4. Fine-tuning Strategy

Standard foundation models are trained for generic tasks (image classification or segmentation) and may not optimally align features for semantic correspondence out-of-the-box. However, full fine-tuning of these large architectures (e.g., ViT-B or ViT-L) is computationally expensive and prone to overfitting on smaller correspondence datasets like SPair-71k.

To balance adaptation with regularization, we adopt a *partial fine-tuning* strategy. We freeze the majority of the backbone parameters and unfreeze only the last $k$ transformer blocks along with the final normalization layers. This allows the model to adapt its high-level semantic abstractions to the specific geometry-aware requirements of keypoint matching while retaining the robust, low-level feature representations learned during pre-training.

**Implementation Details.** We train our models using the AdamW optimizer with a default learning rate of $10^{-5}$ and a weight decay of $0.01$. Given the high resolution of the input images ($518 \times 518$ or similar) and the dense nature of the feature maps, we use only 1 batch. The optimization is driven strictly by the coordinate regression loss defined in Section 4.3, without auxiliary classification heads.

## 5. Experiments

### 5.1. Evaluation Metrics

We evaluate semantic correspondence performance using the standard Percentage of Correct Keypoints (PCK) metric. A predicted keypoint $\hat{\mathbf{p}}$ is considered correct if its Euclidean distance from the ground-truth $\mathbf{p}_{gt}$ falls within a normalized threshold:

$$PCK@\alpha = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\left(||\hat{\mathbf{p}}_i - \mathbf{p}_{gt,i}||_2 \leq \alpha \cdot L_{bbox}\right) \tag{6}$$

where $N$ is the total number of visible keypoints, $L_{bbox}$ is the longer side of the object's bounding box (following the SPair-71k protocol), and $\alpha$ is the threshold factor. We report results at $\alpha = 0.1$ (PCK@0.10).
To assess the efficacy of our coarse-to-fine strategy, we report PCK scores for both:

- **Global PCK:** Calculated using Argmax $\hat{\mathbf{p}}_{global}$ derived from the full image similarity map.
- **Window PCK:** Calculated using the refined prediction $\hat{\mathbf{p}}_{refined}$ with Window Soft-Argmax obtained using the local window search.

The difference between these two metrics directly quantifies the localization gain provided by the window-based refinement module.

### 5.2. Visualizing the features

By visualizing the features generated from the last block in DINOv2, we can clearly see how the model improves the representation of the features to achieve success in the task of semantic correspondence.
In Figure 1, we visualize the PCA-projected embeddings (reduced to 3 dimensions/RGB) for a pair of images. Parts of the image having a similar color indicate a correspondence match in the feature space.

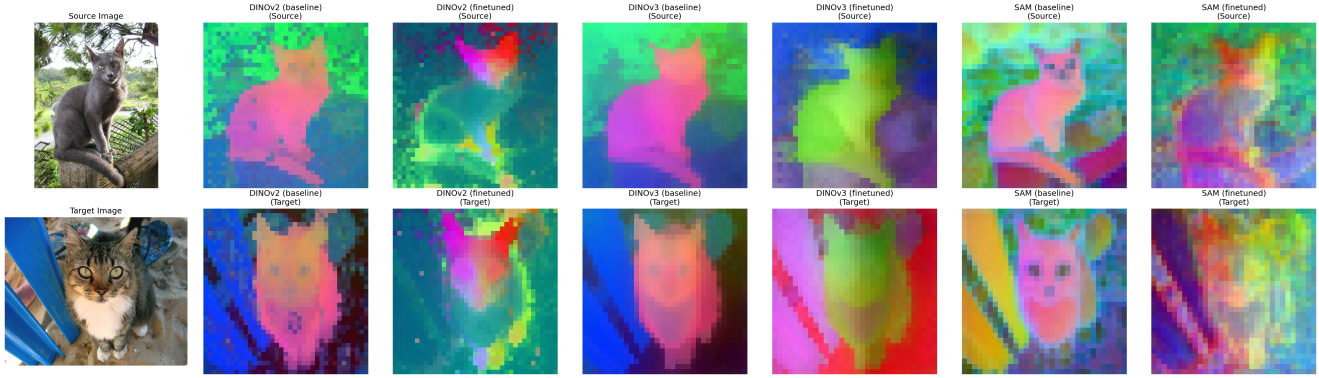
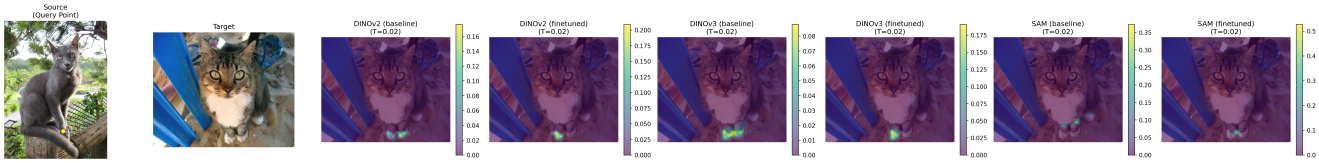
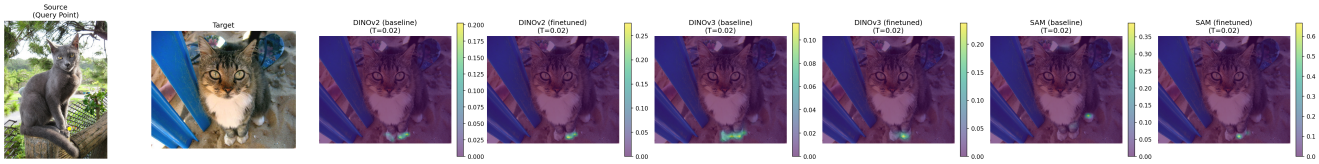



Figure 1. **PCA Feature Visualization (Zero-shot vs. Fine-tuned).** Comparison of feature embeddings from the final transformer block. **(Left)** Pre-trained features lack semantic distinctiveness, with ambiguous coloring between different body parts (e.g., ears vs. paws). **(Right)** After fine-tuning, the model learns a geometry-aware representation where symmetric parts (ears) are consistently matched across images, while distinct semantic regions are sharply separated in feature space.




(a) Right Paw Matching




(b) Left Paw Matching

Figure 2. Matching a point on the right (a) and left (b) paw of a cat. Pretrained model features aren't able to properly distinguish left and right, resulting in both paws having high cosine similarity, while our models exhibit clear distinction, highlighting the single, correct, front paw.

As observed, the **Zero-shot** features (left/top) exhibit a bland color palette with little distinction between body parts; for instance, the cat's ears and paws share similar representations, which leads to potential mismatches. In contrast, the **Fine-tuned** features (right/bottom) show strong semantic consistency. The model explicitly separates concepts: the ears are colored identically across source and target images but are distinct from the paws, ensuring that the model does not mistake one for the other during keypoint matching.

### 5.3. Geometry-Aware features

While Geometry-Aware Semantic Correspondence wasn't our goal, by visualizing heatmap computed with cosine similarity in Figure 2 we can clearly see how our model has improved the ability to distinguish geometric features (left and right paws), even when the keypoints in the source image are near to each other.

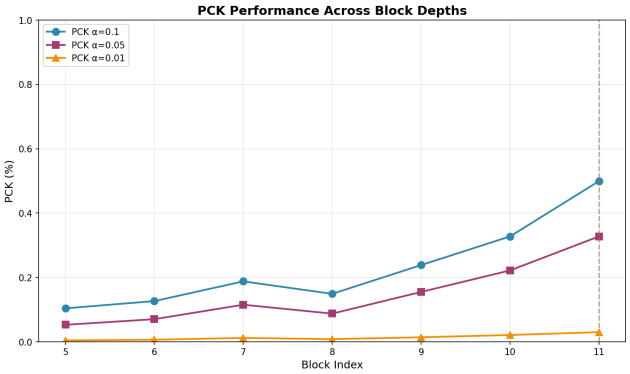
### 5.4. Layer-wise Performance Analysis

To investigate how semantic alignment evolves through the network depth, we evaluate the PCK performance using features extracted from intermediate transformer blocks (specifically blocks 5 through 11). As illustrated in Figure 3b compared to the baseline in Figure 3a, this analysis reveals a critical distinction in feature maturation.

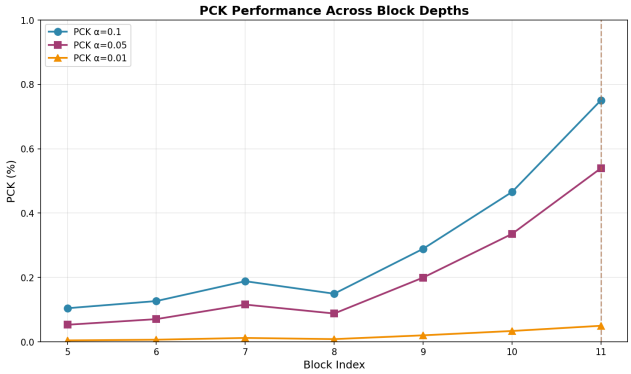
While zero-shot baselines stagnate in deep layers due to generic features, our fine-tuned model demonstrates a dramatic accuracy surge in the final unfrozen blocks. This trajectory confirms that partial fine-tuning successfully specializes deep semantic abstractions into spatially accurate features for dense correspondence.

### 5.5. Pretrained vs. Fine-tuned Performance

We first evaluate the **zero-shot** capabilities of the foundation models, where the entire backbone is frozen and feature maps are used directly for correspondence without task-




(a) Zero-shot DINOv2




(b) Fine-tuned DINOv2

Figure 3. **Layer-wise Performance Analysis.** PCK@0.1 performance at different block depths. **(a)** Zero-shot features stagnate in deeper layers. **(b)** Fine-tuned features show a dramatic accuracy surge in the final unfrozen blocks.

| Category | DINOv2 | DINOv3 | SAM |
|---|---|---|---|
| aeroplane | 85.73 | 74.66 | 67.67 |
| bicycle | 66.74 | 60.58 | 48.94 |
| bird | 93.56 | 87.81 | 75.05 |
| boat | 57.00 | 50.18 | 37.77 |
| bottle | 65.03 | 65.95 | 51.92 |
| bus | 86.71 | 86.44 | 71.71 |
| car | 77.06 | 81.82 | 65.56 |
| cat | 88.64 | 85.71 | 68.65 |
| chair | 72.32 | 53.45 | 41.29 |
| cow | 91.12 | 83.84 | 72.66 |
| dog | 85.96 | 75.05 | 55.64 |
| horse | 82.08 | 75.04 | 59.46 |
| motorbike | 75.51 | 62.55 | 51.76 |
| person | 79.86 | 75.42 | 57.08 |
| pottedplant | 68.11 | 74.67 | 58.19 |
| sheep | 77.00 | 71.50 | 48.92 |
| train | 93.50 | 88.93 | 84.88 |
| tvmonitor | 78.19 | 84.81 | 64.50 |
| **OVERALL (Mean)** | **80.35** | **76.18** | **62.24** |

Table 1. **Evaluation per-category on SPair-71k test set**, PCK@0.10 comparison of our finetuned models. Computed with Window Soft Argmax ($w = 5$). See table 6 for the training parameters.

specific adaptation. As shown in the top section of Table 2, provide a strong starting point, around 50%. Our **partial fine-tuning** strategy (unfreezing the last $k$ blocks) unlocks the semantic potential of these architectures, providing an increase of around 20% across all models. Notably, **DINOv3** achieves the best trade-off, reaching a global PCK of 73.37%, outperforming both DINOv2 and SAM in this semantic correspondence task.

| Model | PCK (kp, g) | PCK (img, g) | Model size (MB) |
|---|---|---|---|
| *Zero-shot Baselines* | | | |
| DINOv2 | **52.72** | **49.94** | 84.20 |
| DINOv3 | 50.71 | 45.67 | 82.52 |
| SAM | 24.00 | 20.62 | 357.67 |
| *Fine-tuned Models* | | | |
| DINOv2 | 71.58 | 68.07 | 84.20 |
| DINOv3 | **73.37** | **69.67** | 82.52 |
| SAM | 61.48 | 57.41 | 357.67 |
| *Fine-tuned with LoRA (rank = 8, alpha = 16)* | | | |
| DINOv2 | **68.50** | **64.68** | 0.57 |
| DINOv3 | 63.55 | 59.81 | 0.57 |
| SAM | 43.80 | 40.44 | 1.50 |
| *Fine-tuned with LoRA (rank = 32, alpha = 64)* | | | |
| DINOv2 | **71.83** | **68.19** | 2.26 |
| DINOv3 | 66.72 | 62.70 | 2.26 |
| SAM | 50.44 | 47.08 | 6.00 |

Table 2. **Quantitative Comparison.** We report PCK@0.10 for baselines, best fine-tuned models and fine-tuned with LoRA (g: global), computed with argmax. When using LoRa, we report the size only of the LoRa weights. Best results are in **bold**.

## 6. Extensions

### 6.1. Data Augmentation

To mitigate overfitting on the small SPair-71k dataset and improve generalization, we implement a *cached* offline augmentation pipeline focused on photometric distortions (color jitter, Gaussian blur, and noise). This approach allows us to scale the effective dataset size while maintaining the computational efficiency of pre-computed feature maps. The quantitative benefit of this strategy is substantial. As detailed in the ablation study (Table 3) for our best-performing model, **DINOv2**, adding augmentation improves the global PCK@0.10 from 71.58% to **78.24%**, a gain of over 6 percentage points.

| Models | DINOv2 | | | DINOv3 | | | SAM | | | DINOv3 + SAM | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| PCK@ $\alpha$ | 0.01 | 0.05 | 0.10 | 0.01 | 0.05 | 0.10 | 0.01 | 0.05 | 0.10 | 0.01 | 0.05 | 0.10 |
| Baseline (Zero-shot) | 3.46 | 35.22 | 52.72 | 3.69 | 32.87 | 50.71 | 1.73 | 13.88 | 24.00 | 4.07 | 33.89 | 51.33 |
| + Fine-tuning | 5.07 | 49.23 | 71.58 | **6.80** | **53.04** | 73.37 | **6.12** | **44.61** | **61.48** | **7.50** | **55.66** | 74.70 |
| + Data Augmentation | **5.65** | **57.10** | **78.24** | 6.36 | 52.05 | **73.51** | 5.85 | 43.85 | 61.11 | 7.47 | 55.10 | **75.02** |
| *Refinement Strategy* | | | | | | | | | | | | |
| Window Soft Argmax ($w = 5$) | 11.11 | 64.05 | 80.24 | **8.57** | **56.63** | 76.50 | **8.57** | **47.97** | 63.21 | **10.34** | **59.78** | 77.81 |
| Window Soft Argmax ($w = 9$) | **11.49** | **65.09** | 81.30 | 7.98 | 55.29 | **77.00** | 7.95 | 47.46 | **64.15** | 9.50 | 58.49 | **78.34** |
| Window Soft Argmax ($w = 15$) | 11.42 | 65.14 | **81.69** | 7.59 | 54.00 | 76.51 | 7.66 | 46.28 | 63.89 | 9.05 | 56.94 | 77.92 |

Table 3. **Ablation Study on SPair-71k.** We analyze the impact of fine-tuning, augmentation, and window refinement sizes on PCK performance. The top section uses standard global Argmax. Best performances are **bold**. Default method is underlined.

Finally, this improvement is most pronounced in the stricter PCK thresholds (PCK@0.05), where DINOv2 jumps from 49.23% to **57.10%**. This indicates that augmenting the training data with diverse visual appearances forces the model to learn more robust, shape-invariant descriptors, resulting in not just more correct matches, but **more precise** localization of keypoints.

Interestingly, this benefit does not generalize to the other architectures like in DINOv3 that goes from 53.04% to 52.05 % with PCK@0.05. This suggests that its specialized Rotary Positional Embeddings (RoPE) or "register" tokens might be sensitive to the disruption of low-level visual cues caused by Gaussian blur and noise. Similarly, **SAM** shows stagnation (61.48% to 61.11%), implying that its mask-based pre-training already encodes a degree of textural invariance that renders simple photometric augmentation redundant. This disparity highlights that while augmentation is critical for standard ViT backbones, more specialized architectures may require tailored strategies to see similar gains.

We also report the per-category results for this models in Table 1.

## 6.2. Mixing DINOv3 and SAM

In our quest to improve the performance of SAM, we try combining it with DINOv3. Looking at the plots from figure 1, we can see how the features learned by SAM have a very different structure than those learned by DINOv3. This happens because SAM was trained for semantic segmentation tasks, while DINOv3 is a more general model, oriented to multiple tasks. To combine the two models, we pass the image to both models, extracting the features $F_{\text{SAM}}$ and $F_{\text{DINOv3}}$, then we normalize and concatenate the two features, adding a manual weight hyperparameter. We use $w_{\text{SAM}} = 0.3$, $w_{\text{DINOv3}} = 0.7$ (Table 4) returning a single feature map $F_{\text{DINOv3+SAM}} = \text{Concat}(w_{\text{DINOv3}} F_{\text{DINOv3}}, w_{\text{SAM}} F_{\text{SAM}})$. The results show a big improvement over the best finetune of SAM, and a mild improvement over our best finetuned DINOv3 (around +2%), as can be seen in table 3. We provide results using, in pair, the baselines DINOv3+SAM, the finetuned DINOv3+SAM without data augmentation, and the finetuned DINOv3+SAM with data augmentation, according to table 3.

| **Weights** | | **PCK Threshold** ($\alpha$) | | |
|---|---|---|---|---|
| $w_{\text{DINOv3}}$ | $w_{\text{SAM}}$ | 0.01 | 0.05 | 0.10 |
| 0.1 | 0.9 | 7.02 | 43.86 | 61.63 |
| 0.2 | 0.8 | 7.48 | 45.36 | 63.45 |
| 0.3 | 0.7 | 9.73 | 52.60 | 68.41 |
| 0.4 | 0.6 | 10.38 | 56.22 | 72.30 |
| 0.5 | 0.5 | **11.03** | 59.25 | 75.76 |
| 0.6 | 0.4 | 10.99 | **60.58** | 77.62 |
| 0.7 | 0.3 | 10.34 | 59.78 | **77.81** |
| 0.8 | 0.2 | 9.29 | 58.28 | 77.19 |
| 0.9 | 0.1 | 8.69 | 56.96 | 76.59 |

Table 4. **Mixing Weights Analysis.** PCK computed with Window Soft Argmax ($w = 5$) performance at different thresholds ($\alpha$) for linear combinations of DINOv3 and SAM features. Best performances are **bold**.

## 6.3. Additional Backbones

To explore the boundaries of our method, we extend our evaluation to two extreme ends of the spectrum: mobile-grade efficiency and large-scale model size. This allows us to perform two key comparisons: evaluating the impact of pre-training objectives on identical lightweight architectures (TinyViT vs. TinySAM) and assessing the scalability of semantic knowledge (TinySAM vs. SAM ViT-Large).

**Tiny Architectures: TinyViT vs. TinySAM.** We first compare two lightweight models designed for resource-constrained scenarios. Interestingly, these models share a similar hierarchical architecture but differ fundamentally in

their pre-training supervision.

- **TinyViT:** We utilize the `TinyViT-21M` model [7], which employs a hierarchical structure with depth-wise convolutions and windowed attention. It is supervised on ImageNet-21k, optimizing primarily for object classification.
- **TinySAM:** We employ a mobile-optimized variant of SAM [4]. While it utilizes a similar lightweight encoder backbone, its weights are obtained via knowledge distillation from the heavy SAM ViT-H teacher.

Comparing these two allows us to isolate the effect of the *training objective*: does the classification-based ImageNet pre-training or the geometry-aware SAM distillation yield better features for dense correspondence?

As shown in Table 5, TinySAM demonstrates superior geometric awareness out-of-the-box, achieving a Zero-shot PCK of 21.47% compared to TinyViT's 13.82%. This confirms that distillation from a segmentation teacher yields better initial dense features than classification pre-training. However, TinyViT proves significantly more adaptable: it reaches 49.84% after fine-tuning, surpassing TinySAM (36.30%). This suggests that while distilled features are semantically richer initially, the classification-based weights of TinyViT offer greater elasticity for task-specific adaptation.

**Scaling Model Size: TinySAM vs. SAM ViT-Large.** To determine if "bigger is strictly better" for semantic correspondence, we compare the lightweight TinySAM against the massive SAM ViT-Large.

- **SAM ViT-Large:** This model scales the standard ViT-Base encoder to 24 transformer blocks with an embedding dimension of 1024, resulting in approximately 307 million parameters.

We hypothesize that while the Large model captures subtler semantic nuances, the lightweight models might offer competitive efficiency. The results support this in the zero-shot setting, where TinySAM (21.47%) is remarkably close to SAM ViT-Large (23.35%), despite having $14\times$ fewer parameters as we shown in Table 5. However, capacity proves critical for fine-tuning performance. Interestingly, we observe diminishing returns at the extreme scale: SAM ViT-Base achieves the highest overall score of 57.64%, outperforming the substantially larger ViT-Large (54.19%), indicating that the largest models may be harder to optimize or prone to overfitting for this specific task.

| Model | # Parameters | Zero-shot | Fine-tuned |
|---|---|---|---|
| *Tiny model size* | | | |
| TinyViT-21M | 21M | 13.82 | 49.84 |
| TinySAM | 21M | 21.47 | 36.30 |
| *Base model size* | | | |
| SAM ViT-Base | 91M | 24.71 | 57.64 |
| *Large model size* | | | |
| SAM ViT-Large | 307M | 23.35 | 54.19 |

Table 5. **Performance vs. Size Evaluation.** Comparison of PCK@0.10 scores per keypoint, computed with global argmax, across lightweight and large-scale backbones. All backbones have been trained by unfreezing the last 3 blocks.

### 6.4. Low-rank adaptation (LoRA)

To enable efficient fine-tuning of large foundation models without the high computational cost of updating all parameters, we implement Low-Rank Adaptation (LoRA). Instead of full fine-tuning, LoRA freezes the pre-trained model weights $W_0$ and injects trainable rank decomposition matrices $A$ and $B$ into each layer, such that $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$.

**Implementation Strategy.** We integrate LoRA adapters directly into specific architectural components of the backbone models. Crucially, we do not apply LoRA uniformly across the entire depth of the network. Consistent with our partial fine-tuning strategy, we target only the last $N$ transformer blocks to preserve the generic low-level features while adapting the high-level semantic representations.

We configure LoRA with a rank of $r = 8$ and a scaling factor $\alpha = 16$. The specific target modules differ by backbone to ensure maximum adaptation flexibility:

- **DINOv2 / DINOv3:** We target the projection matrices within the Attention mechanisms (specifically the Query-Key-Value `attn.qkv` and output `attn.proj` projections) as well as the dense layers of the Feed-Forward Networks (`mlp.fc1`, `mlp.fc2`) within each block.
- **SAM:** In addition to the attention and MLP layers, we extend LoRA to the feature pyramid `neck` module. This component is responsible for compressing the encoder features before they reach the task-specific heads, making it a critical adaptation point for semantic correspondence in our architecture.

By updating only these low-rank matrices and the normalization layers, we reduce the number of trainable parameters by orders of magnitude while maintaining performance comparable to full fine-tuning.

**Results.** Table 2 presents the effectiveness of parameter-efficient fine-tuning compared to our standard partial fine-tuning strategy. We report results for two LoRA configurations: a standard setting (rank=8, alpha=16) and a higher-capacity setting (rank=32, alpha=64). Our experiments reveal that the capacity of the adapter plays a crucial role in semantic correspondence. Increasing the rank from 8 to 32 resulted in a clear performance gain across all architectures. For instance, DINOv2 improved from 68.50% to 71.83%,

| Configuration | DINOv2 | DINOv3 | SAM |
|---|---|---|---|
| *Training* | | | |
| Learning Rate ($lr$) | $1 \times 10^{-5}$ | $1 \times 10^{-5}$ | $5 \times 10^{-5}$ |
| Optimizer | AdamW | AdamW | AdamW |
| Effective Batch Size | 1 | 1 | 1 |
| Grad. Accum. Steps | 4 | 4 | 4 |
| Total Epochs | 10 | 10 | 8 |
| *Architecture & Extensions* | | | |
| Unfrozen Blocks ($k$) | 3 | 3 | 4 |
| Data Augment. (Ext) | 7 images | 7 images | 7 images |
| Weight Decay | 0.01 | 0.01 | 0.01 |
| Temperature | 0.1 | 0.02 | 0.02 |

Table 6. **Chosen Models Configurations.** Detailed hyperparameters for the best PCK performing checkpoint of each architecture.

and SAM saw a recovery of approximately 6 percentage points (43.80% to 50.44%). This improvement confirms the hypothesis that the "bottleneck" created by a low-rank constraint was partly responsible for the performance drop; the model required more trainable parameters to learn the complex spatial mappings needed for this task.

However, even with the quadrupled rank, LoRA still fails to match the performance of standard fine-tuning with some models. On SAM remains substantial at over 10 points. This suggests that the transformation required to adapt these foundation models for dense semantic correspondence is inherently high-rank and cannot be fully captured by a low-rank decomposition, even at rank 32.

Despite the lower accuracy, LoRA offers a massive advantage in storage efficiency. As shown in the "Model size" column, the standard DINOv2 stores 84.20 MB of weights, whereas the LoRA adapters require only 0.57 MB (rank=8) or 2.26 MB (rank=32). This represents a size reduction of over 97%. While standard fine-tuning is necessary to achieve state-of-the-art accuracy, LoRA provides a viable alternative for highly constrained environments where storage is critical, followed by a drop in precision.

### 6.5. Conclusions

In this work we have shown how effective is finetuning transformer based vision models to improve their performance on semantic correspondence tasks. We show that finetuning bigger models, with the same number of unfrozen blocks, doesn't necessarily provide a PCK score improvement. We additionally show how data augmentation, and matching keypoints using window softargmax, are providing a noticeable improvement. Finally, we show finetuning with LoRa provides an improvement over the baseline, requiring us to save only 0.57MB of additional weights.

## References


[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 1

[2] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023. 1

[3] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Haziza Daniel, Babouchesku Adina, Couprie Marek, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 1

[4] Han Shu, Wenshuo Li, Yehui Tang, Yiman Zhang, Yihao Chen, Houqiang Li, Yunhe Wang, and Xinghao Chen. Tinysam: Pushing the envelope for efficient segment anything model, 2025. 7

[5] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. Dinov3, 2025. 1

[6] Luming Tang, Menglin Jia, Qianqian Wang, Cheng Perng Phoo, and Bharath Hariharan. Emergent correspondence from image diffusion, 2023. 1, 2

[7] Kan Wu, Jinnian Zhang, Houwen Peng, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. Tinyvit: Fast pretraining distillation for small vision transformers, 2022. 7

[8] Junyi Zhang, Charles Herrmann, Junhwa Hur, Luisa Polania Cabrera, Varun Jampani, Deqing Sun, and Ming-Hsuan Yang. A tale of two features: Stable diffusion complements dino for zero-shot semantic correspondence. *Advances in Neural Information Processing Systems*, 36:45533–45547, 2023. 2

[9] Junyi Zhang, Charles Herrmann, Junhwa Hur, Eric Chen, Varun Jampani, Deqing Sun, and Ming-Hsuan Yang. Telling left from right: Identifying geometry-aware semantic correspondence, 2024. 1, 2, 3